

In einem verfahrenstechnischen Prozess bei einem Lebensmittelhersteller werden in einem Rührwerk Zutaten gemischt, wobei die Temperatur, die Luftfeuchtigkeit und der Füllstand innerhalb des Rührwerks als Qualitätsparameter kontinuierlich gemessen werden. Sie erhalten den Auftrag diese Parameter zu protokollieren und nachweisfähig auf einem Cloud-Datenspeicher zu archivieren sowie die Konfiguration der ausgewählten Plattform und den automatisierten Dateizugriff über deren API zu planen. Danach sollen Sie die periodische Erfassung der Sensordaten in einem Standarddateiformat auf einem zugehörigen Einplatinenrechner/Microcontrollerboard implementieren. Ebenso wird die regelmäßige automatische Übertragung der Log-Daten auf den Cloud-Speicher erfasst.

Sie erhalten die Anweisung ihre Implementierung in einer Laborumgebung zu testen, die die Temperatur im Rührwerk des Lebensmittelherstellers nachbilden kann und die entsprechende Sensorik sowie das erforderliche Erfassungssystem enthält. Dabei soll besonderes Augenmerk auf die Funktionalität der automatisierten Messdatenerfassung und die API basierte Kommunikation mit der Cloud-Plattform gelegt werden.



Abb.1: Mischanlage

Informationsbeschaffung:

Damit Sie den Auftrag realisieren können sollen Sie sich zu diesem Zweck über **Public-, Hybrid- bzw. Private-Cloud-Plattformen**, die für die vorgesehene Datenhaltung auch unter Datensicherheitsaspekten geeignet sind, informieren und deren Vor- und Nachteile tabellarisch festhalten. Klären Sie weiterhin die Servicemodelle **IaaS** (Infrastructure as a Service), **PaaS** (Platform as a Service) sowie **SaaS** (Software as a Service) und nennen Sie je ein Beispiel für jeden Service.

Teil 1:

Sie haben sich zusammen mit dem Kunden für die Verwendung von Proxmox, einem Typ-1-Hypervisor, entschieden. Auf diesem sollen zukünftig verschiedene Dienste in sogenannten LXC-Systemcontainern bzw. in virtuellen Maschinen laufen. Als Grundlage des aktuell zu realisierenden Projektes soll eine Debian12-Instanz dienen, welche entsprechend konfiguriert werden muss. **Weiterhin sollen Sie ein Protokoll für die Konfigurationsschritte erstellen!**

- Loggen Sie sich über die URL <https://bszwsz.prox-fi.ipv64.net:2627> als user „**schule**“ mit dem Passwort „**BszWsw25!!**“ unter der Domain „**Proxmox VE authentication server**“ ein und erstellen Sie einen unprivilegierten LXC-Systemcontainer mit einem Debian12 Image. Vergeben Sie auch einen eindeutigen Hostnamen sowie ein Kennwort.
- Der verwendete Festplattenpeicher für den Container soll eine Größe von 10 GiB besitzen sowie 2 CPU Kerne. Für den Arbeitsspeicher wählen Sie eine Größe von 1024 MiB.
- Konfigurieren Sie die Netzwerkeinstellungen so, dass Sie die vorhandene Netzwerkbrücke „vbr0“ verwenden. Die jeweilige IPv4-Adresse entnehmen Sie der Excel-Datei. Eine IPv6-Adresse benötigen Sie nicht.
- Die DNS Domain und der DNS Server bekommen ihre Werte vom Host.
- Schließen Sie zum Schluss die Konfiguration unter „Bestätigen“ ab.
- Der Container soll dabei noch nicht gestartet werden, da nach der Konfiguration unter der Container-Option „Features“ zuerst noch ein Haken unter „**keyctl**“ gesetzt werden muss (wird im weiteren Verlauf für Docker benötigt). Starten Sie den Container erst, **nachdem ihr Lehrer** dieses Feature aktiviert hat (root user only).

Nachdem Ihre erstellte Instanz läuft, führen Sie den Befehl „**apt update && apt upgrade -y**“ aus. Falls Ihnen die Befehle der Linux Shell fremd sind, informieren Sie sich dementsprechend. Es ist für die weitere Arbeit notwendig, dass Sie in Ihrer erstellten Instanz **NodeRed** und den MQTT Broker **mosquitto** installieren. Beides soll dabei als **Service** ausgeführt werden, sowie dass der MQTT-Broker und NodeRed **passwortgeschützt** sein sollen.

Der Nginx-Systemcontainer:

Da für den späteren Verlauf sowohl NodeRed als auch mosquitto über **https** aufrufbar sein soll, müssen Sie sich um die notwendigen Zertifikate für die Transport Layer Security (TLS) kümmern. Für dessen Relevanz können Sie eine beliebige https-Seite aufrufen und im Browser die Zertifikatsbestandteile weitergehend untersuchen und unbekannte Begriffe klären. Für die Sicherheitstechnische Realisierung wurde im Nginx-Container bereits die Software letsencrypt und certbot installiert.

Zusatzaufgabe:

1. Die hohe Verfügbarkeit (24/7) eines Webserver ist in der Realität geschäftsentscheidend. **Erläutern Sie hierzu die beiden Schlüsselmetriken für Ausfallsicherheit im Cloud-Betrieb (RTO – Recovery Time Objective; RPO – Recovery Point Objective).**
2. Der Webserver wird ausschließlich HTTPS-Verbindungen erlauben. Erläutern Sie zwei Aufgaben einer Certificate Authority (CA).
3. Erläutern Sie den schrittweisen Ablauf eines Zertifikatshandshakes beim Verbindungsaufbau eines Clients zu Ihrem Webserver.
4. Erläutern Sie die Prinzipien eines Load Balancers sowie die Begriffe Round Robin, Least Connections und IP-Hashing.
5. Erläutern Sie das Angriffsszenario eines DDoS-Angriffs und wie der Webserver gegen solch einen Angriff abgesichert werden könnte.

Da Nginx in diesem Fall als Reverse Proxy fungiert, sollen Sie für das Projekt die dementsprechende Konfiguration realisieren. Dabei sollen Ihnen sowohl folgende Codeschnipsel als auch folgende Seite helfen:

<https://www.hostinger.com/tutorials/how-to-set-up-nginx-reverse-proxy/>

NodeRed:

```
server {
    server_name <YOUR_DNS>;

    location / {
        proxy_pass http://<YOUR_IPv4>:1880;
        # WebSocket Support
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        #
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

mosquito:

```
server {
    server_name <YOUR_DNS>;

    location / {
        proxy_pass http://<YOUR_IPv4>:1883;
        # WebSocket Support
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        #
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```



Um eine Domain sowie eine Subdomain zu erhalten, können Sie sich zum Beispiel bei <https://ipv64.net> einen kostenlosen Account erstellen, um dann einen A-Record für Ihren Reverse Proxy erstellen zu können. Beachten Sie hierbei, dass Sie die Public IPv4 des Hostsystems benötigen.

Um schließlich ein gültiges Zertifikat anzufordern, nutzen Sie folgenden Befehl:
certbot -- nginx -d <YOUR_DOMAIN>

Überprüfen Sie anschließend unter `/etc/letsencrypt/live/<YOUR_DOMAIN>/` die Bestandteile der beiden generierten Zertifikate und überprüfen Sie die Erreichbarkeit Ihrer erstellten NodeRed-Instanz (die Domain für mosquito benötigen Sie für die Übermittlung der Daten Ihres CPS's an benannte Instanz).

ACHTUNG:

Da lediglich die Existenz eines Nginx-Containers möglich ist, welcher Zertifikate über den Port 80 beziehen kann und Sie über die Proxmox Web GUI nur einzeln für Bearbeitungen auf jenen Container zugreifen können, sollen Sie die Konfiguration des Reverse Proxy über eine SSH-Verbindung aufbauen! Sie können dazu zum Beispiel folgende Seite nutzen:

<https://ssheasy.com>

Es empfiehlt sich allerdings der Zugriff direkt von Ihrem persönlichen Endgerät über zum Beispiel PowerShell!

Weitere Hilfestellung:

- <https://nodered.org/docs/getting-started/raspberrypi>
- <https://www.vultr.com/docs/install-mosquitto-mqtt-broker-on-ubuntu-20-04-server/>

Testen Sie in der NodeRed Entwicklungsumgebung Ihren installierten MQTT-Broker auf Funktionalität! **Erstellen Sie ein Protokoll, in welchem Sie die notwendigen Konfigurationsschritte festhalten!**

Cyberphisches System:

Erstellen Sie ein cyberphisches System auf der Grundlage eines ESP32 und erzeugen Sie ein dementsprechendes Programm in Micropython, um die notwendigen Sensordaten an Ihren MQTT-Broker senden zu können. Führen Sie einen Test durch, um Ihre Ergebnisse zu validieren.